

TiDB Cloud Serverless

One brain for every AI coding tool

A shared, queryable infrastructure knowledge graph on TiDB that Claude Code, Codex, and Cursor all read and write through MCP - so AI-written infrastructure stops duplicating work, breaking conventions, and starting from zero.

Claude Code

Codex

Cursor

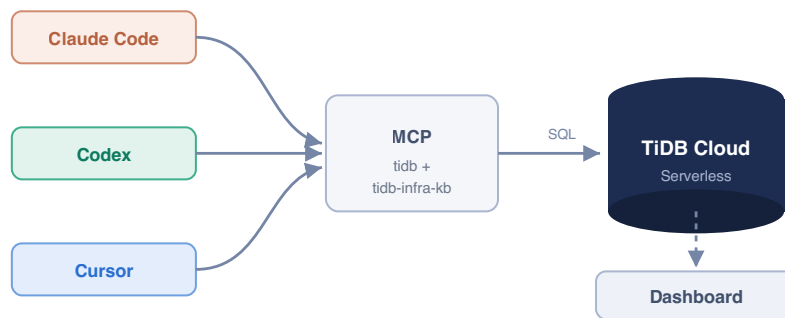
Model Context Protocol

Recursive CTE graph

What this is

Teams now write infrastructure with AI coding tools. But each tool works **blind and alone** - it cannot see what already exists, the team's conventions, or how components connect. The result is duplication, drift, broken conventions, and outages.

This demo puts the engineering knowledge where every tool can use it: a **knowledge graph in TiDB Cloud Serverless**. Every coding tool connects to it over the **Model Context Protocol (MCP)**, queries it before building, writes its work back, and traverses component relationships with **recursive SQL**. The example workload is an infrastructure-as-code monorepo (Pulumi / TypeScript) for a fictional company, *Acme*, spanning AWS, Cloudflare, and Okta.



Three real CLIs → two MCP servers → one TiDB database. The dashboard reads the same database live.

Proven on real TiDB. Everything in this report - the graph, the writes from three tools, and the recursive-CTE traversals - ran against a live TiDB Cloud Serverless cluster (v8.5.3) over the MySQL protocol with TLS.

The problem: AI tools with no shared memory

Four failure modes show up the moment more than one agent (or session) touches the same codebase.

FAILURE MODE	WHAT HAPPENS WITHOUT A SHARED GRAPH
Duplication	A tool can't see a component already exists, so it builds a second one. Drift and double cost follow.
Convention breaks	It writes a raw <code>aws.s3.Bucket</code> instead of the approved library wrapper, skipping required tags and policies.
Invisible blast radius	It changes a shared library and breaks resources two hops away - discovered only in production.
Cold starts	The next tool or session re-derives everything from the filesystem and re-makes the same mistakes.

The common root cause: the knowledge that prevents these - **what exists, how it connects, and the rules** - lives only in scattered code and people's heads. It is not queryable, and it is not shared across tools.

The fix. Model the codebase as a graph in TiDB: *infra_components* (nodes), *component_edges* (typed relationships), and *session_log* (who did what). Expose it to every tool over MCP. Now the tools query before they build and hand off warm.

The live demo: three tools, one task

The task: production is complete, but **staging is missing its DNS and SSO layer**. Watch three different AI coding tools finish it - each picking up where the last left off, because they share one TiDB brain.

The screenshot displays the 'TiDB Infra Knowledge Base' interface. The top navigation bar includes 'Live', 'Dependency graph (CTE)', and 'Why TiDB - scenarios'. The main content area is divided into two panels.

Left Panel: 3 tools → 1 TiDB

Every query and write from Claude Code, Codex, and Cursor lands in the same TiDB database. Watch them collaborate.

- seed** Created S3 component 'acme-staging-data-exports' in staging 14:27:51
- WRITE**
- seed** Created RDS component 'acme-staging-analytics-db' in staging 14:27:49
- WRITE**
- seed** Created Okta component 'acme-prod-admin-ss0' in production 14:27:46
- WRITE**
- seed** Created Cloudflare component 'acme-prod-admin-dns' in production 14:27:43
- WRITE**
- seed** Created Cloudflare component 'acme-prod-assets-dns' in production 14:27:40
- WRITE**
- seed** Created S3 component 'acme-prod-static-assets' in production 14:27:38
- WRITE**
- seed** Created S3 component 'acme-prod-data-exports' in production 14:27:35
- WRITE**
- seed** Created RDS component 'acme-prod-analytics-db' in production 14:27:32
- WRITE**
- seed** Created Library component 'SsoApplication' in library 14:27:30
- WRITE**
- seed** Created Library component 'DnsRecord' in library 14:27:27
- WRITE**

Right Panel: Shared knowledge graph

The graph shows relationships between components across different environments (All, Lib, Prod, Staging). Nodes include 'analytics-db', 'data-exports', 'admin-ss0', 'admin-dns', 'assets-dns', and 'DnsRecord'. Edges represent relationships like 'uses', 'redirects_to', and 'instances'.

Staging parity vs production

4 missing in staging

Component	Provider	Status
admin-dns	Cloudflare	missing
assets-dns	Cloudflare	missing
admin-ss0	Okta	missing
analytics-db	RDS	in staging
data-exports	S3	in staging
static-assets	S3	missing

Starting state. Left: the live activity feed (one row per query/write from any tool). Right: the knowledge graph and the staging-parity strip - four components are missing in staging (red X).

Step 1 - Claude Code inspects, then builds the asset layer claude-code

It first **queries** the graph (it does not assume), then scaffolds the missing static-assets bucket and its Cloudflare DNS record by composing the existing *S3Bucket* and *DnsRecord* libraries - never raw cloud resources - and **writes each one back**.

tidb

Infra Knowledge Base · TIDB Cloud Serverless

Claude Code

Codex

Cursor

→ one TiDB

Reset KB

Live

Dependency graph (CTE)

Why TiDB · scenarios

3 tools → 1 TiDB

Every query and write from Claude Code, Codex, and Cursor lands in the same TiDB database. Watch them collaborate.

claude-code

Created Cloudflare component 'acme-staging-assets-dns' in staging

14:28:26

WRITE

claude-code

Created S3 component 'acme-staging-static-assets' in staging

14:28:23

WRITE

claude-code

SELECT environment, name, component_type FROM infra_components WHERE environment IN ('staging','production') ORDER BY en

14:28:20

QUERY

seed

Created S3 component 'acme-staging-data-exports' in staging

14:27:51

WRITE

seed

Created RDS component 'acme-staging-analytics-db' in staging

14:27:49

WRITE

seed

Created Okta component 'acme-prod-admin-ssso' in production

14:27:46

WRITE

seed

Created Cloudflare component 'acme-prod-admin-dns' in production

14:27:43

WRITE

seed

Created Cloudflare component 'acme-prod-assets-dns' in production

14:27:40

WRITE

seed

Created S3 component 'acme-prod-static-assets' in production

14:27:38

WRITE

seed

Created S3 component 'acme-prod-data-exports' in production

14:27:35

WRITE

Shared knowledge graph

All

Lib

Prod

Staging

Library

S3

RDS

Cloudflare

Okta

Staging parity vs production

2 missing in staging

✗	admin-dns	Cloudflare	missing
✓	assets-dns	Cloudflare	in staging
✗	admin-ssso	Okta	missing
✓	analytics-db	RDS	in staging
✓	data-exports	S3	in staging
✓	static-assets	S3	in staging

Two new components written by cClaude-code (green WRITE rows); two staging gaps close.

Step 2 - Codex picks up warm codex

Codex opens with **zero local memory** of Claude Code's work. It reads the shared `session_log`, sees exactly what was just created, and continues with the admin-portal DNS record. No copy-pasting context between tools.

tidb

Infra Knowledge Base · TIDB Cloud Serverless

Claude Code

Codex

Cursor

→ one TiDB

Reset KB

Live

Dependency graph (CTE)

Why TiDB · scenarios

3 tools → 1 TiDB

Every query and write from Claude Code, Codex, and Cursor lands in the same TiDB database. Watch them collaborate.

codex

WRITE

Created Cloudflare component 'acme-staging-admin-dns' in staging

14:28:52

codex

QUERY

SELECT developer, action, detail FROM session_log ORDER BY created_at DESC LIMIT 4;

14:28:49

claude-code

WRITE

Created Cloudflare component 'acme-staging-assets-dns' in staging

14:28:26

claude-code

WRITE

Created S3 component 'acme-staging-static-assets' in staging

14:28:23

claude-code

QUERY

SELECT environment, name, component_type FROM infra_components WHERE environment IN ('staging','production') ORDER BY en

14:28:20

seed

WRITE

Created S3 component 'acme-staging-data-exports' in staging

14:27:51

seed

WRITE

Created RDS component 'acme-staging-analytics-db' in staging

14:27:49

seed

WRITE

Created Okta component 'acme-prod-admin-ssso' in production

14:27:46

seed

WRITE

Created Cloudflare component 'acme-prod-admin-dns' in production

14:27:43

seed

WRITE

Created Cloudflare component 'acme-prod-assets-dns' in production

14:27:40

Shared knowledge graph

All

Lib

Prod

Staging

Library

S3

RDS

Cloudflare

Okta

Staging parity vs production

1 missing in staging

✓	admin-dns	Cloudflare	in staging
✓	assets-dns	Cloudflare	in staging
✗	admin-ssso	Okta	missing
✓	analytics-db	RDS	in staging
✓	data-exports	S3	in staging
✓	static-assets	S3	in staging

A codex QUERY reads the session log (blue), then a WRITE adds the admin DNS - continuing the same task seamlessly.

Step 3 - Cursor verifies with a recursive CTE, then finishes cursor

Before writing the SSO application, Cursor runs a **recursive query** over the graph to confirm the dependency pattern (an SSO app must redirect to a DNS record), then scaffolds it correctly. Staging reaches parity.

tidb

Infra Knowledge Base

TiDB Cloud Serverless

Claude Code

Codex

Cursor

→ one TiDB

Reset KB

Live

Dependency graph (CTE)

Why TiDB · scenarios

3 tools → 1 TiDB

Every query and write from Claude Code, Codex, and Cursor lands in the same TiDB database. Watch them collaborate.

cursor

WRITE

Created Okta component 'acme-staging-admin-sso' in staging

14:29:16

cursor

QUERY

-- What does 'acme-prod-admin-sso' transitively depend on? (downstream traversal) WITH RECURSIVE deps(from_id, to_id, re

14:29:13

codex

WRITE

Created Cloudflare component 'acme-staging-admin-dns' in staging

14:28:52

codex

QUERY

SELECT developer, action, detail FROM session_log ORDER BY created_at DESC LIMIT 4;

14:28:49

claude-code

WRITE

Created Cloudflare component 'acme-staging-assets-dns' in staging

14:28:26

claude-code

WRITE

Created S3 component 'acme-staging-static-assets' in staging

14:28:23

claude-code

QUERY

SELECT environment, name, component_type FROM infra_components WHERE environment IN ('staging','production') ORDER BY en

14:28:20

seed

WRITE

Created S3 component 'acme-staging-data-exports' in staging

14:27:51

seed

WRITE

Created RDS component 'acme-staging-analytics-db' in staging

14:27:49

seed

WRITE

Created Okta component 'acme-prod-admin-sso' in

14:27:46

Shared knowledge graph

All

Lib

Prod

Staging

Library S3 RDS Cloudflare Okta

Staging parity vs production

at parity ✓

✓	admin-dns	Cloudflare	in staging
✓	assets-dns	Cloudflare	in staging
✓	admin-sso	Okta	in staging
✓	analytics-db	RDS	in staging
✓	data-exports	S3	in staging
✓	static-assets	S3	in staging

All three tools' activity is visible in one feed. The parity strip is now fully green: staging matches production, built by three tools sharing one memory.

The query a vector database can't answer

The headline capability: **graph reachability**. "What breaks if I change this component?" requires traversing dependency edges to arbitrary depth. A recursive CTE does it in one query.

tidb

Infra Knowledge Base · TiDB Cloud Serverless

Claude Code

Codex

Cursor

→ one TiDB

Reset KB

Live

Dependency graph (CTE)

Why TiDB · scenarios

The question a flat repo or a vector store can't answer: what transitively depends on this component? A recursive CTE traverses the dependency graph to any depth. This exact SQL runs on the official [tidb](#) MCP that the CLIs use.

Component

S3Bucket

Blast radius (what breaks if this changes)

Dependencies (what this needs)

Run recursive CTE

Traversal graph

S3Bucket · 10 nodes

depth 1 depth 2 depth 3+

The query (SQLite + TiDB)

```
-- What breaks if 'S3Bucket' changes? (upstream blast-r
WITH RECURSIVE blast(from_id, to_id, relationship, dept
  SELECT e.from_id, e.to_id, e.relationship, 1
  FROM component_edges e
  JOIN infra_components c ON e.to_id = c.id
  WHERE c.name = 'S3Bucket'
 UNION ALL
  SELECT e.from_id, e.to_id, e.relationship, b.depth
  FROM component_edges e
  JOIN blast b ON e.to_id = b.from_id
  WHERE b.depth < 10
)
SELECT DISTINCT b.depth, cf.name AS from_name,
  b.relationship, ct.name AS to_name,
  cf.component_type, cf.environment
FROM blast b
JOIN infra_components cf ON b.from_id = cf.id
JOIN infra_components ct ON b.to_id = ct.id
ORDER BY b.depth, from_name;
```

Result rows

9 rows

d1	PostgresDatabase	--instantiates→	S3Bucket
d1	data-exports	--uses→	S3Bucket
d1	static-assets	--uses→	S3Bucket
d1	data-exports	--uses→	S3Bucket
d1	static-assets	--uses→	S3Bucket
d2	analytics-db	--uses→	PostgresDatabase
d2	assets-dns	--fronts→	static-assets
d2	analytics-db	--uses→	PostgresDatabase
d2	assets-dns	--fronts→	static-assets

Blast radius of the S3Bucket library, visualised as a depth-coloured subgraph. The traversal reaches the RDS databases **two hops away** - their backup buckets compose S3Bucket - a connection no text search could surface.

```
-- What breaks if S3Bucket changes? Runs unchanged on SQLite and TiDB.
WITH RECURSIVE blast(from_id, to_id, relationship, depth) AS (
  SELECT e.from_id, e.to_id, e.relationship, 1
  FROM component_edges e JOIN infra_components c ON e.to_id = c.id
  WHERE c.name = 'S3Bucket'
 UNION ALL
  SELECT e.from_id, e.to_id, e.relationship, b.depth + 1
```



```
FROM component_edges e JOIN blast b ON e.to_id = b.from_id
WHERE b.depth < 10
)
SELECT depth, from_id, relationship, to_id FROM blast ORDER BY depth;
```

Why a vector store can't do this. Vector similarity finds *related-looking* text. It cannot compute a transitive closure over a dependency graph. This is reachability - exactly what SQL recursion is built for - and TiDB runs it as a distributed query alongside the same tables that hold the components, their tags, and (optionally) their vector embeddings.

Why TiDB is the right brain for agents

In the emerging pattern, the database is the agent's persistent memory, execution-state store, and shared knowledge base. That demands more than a vector index.

One engine, every access pattern

Agents need **SQL** for structured state, **graph traversal** for relationships, **vector search** for semantic memory, and **full-text (BM25)** for knowledge - all over the same data, with no ETL and no sync lag. TiDB provides all of them in one MySQL-compatible engine. This demo uses SQL + recursive CTEs today; the same *infra_components* table can carry a *VECTOR* column for hybrid graph-plus-semantic retrieval with *VEC_COSINE_DISTANCE*.

ACID for agent state

Each write here is one transaction: **component + dependency edge + audit-log entry, all or nothing**. TiDB's distributed ACID prevents the partial-write bugs that make multi-agent systems nearly impossible to debug.

Built for agent scale

Serverless, pay-per-request, scales to zero. Millisecond database branching gives each agent or workspace an isolated environment - the pattern behind production fleets running millions of agent databases on TiDB Cloud.

How it compares for this workload

CAPABILITY	TIDB CLOUD	VECTOR DB (PINECONE)	POSTGRES / PGVECTOR
Recursive graph traversal (CTE)	✓	✗	✓ (single node)
SQL + JOINS + ACID	✓	✗	✓
Vector + full-text in one engine	✓	vector only	partial
Horizontal scale / millions of tenants	✓	partial	✗

Proof at scale: a leading agent platform runs 10M+ databases and billions of events/day on TiDB Cloud, with 100% of those databases created by agents.

Concrete scenarios the graph prevents

Two everyday infrastructure mistakes, and how a queryable graph stops them before code is written.

The duplicate backup bucket

cursor

HEADLINE

Task: Add backups for the staging analytics database.

WITHOUT THE GRAPH

- ✗ Dev greps the repo, finds no backup bucket for analytics.
- ✗ Writes a raw `new aws.s3.BucketV2('analytics-backup')`.
- ✗ Two bugs ship: (1) a DUPLICATE bucket - PostgresDatabase already makes one;
- ✗ (2) a raw provider resource, violating the no-raw-resources rule.
- ✗ Drift, double storage cost, and an untagged bucket nobody owns.

WITH TiDB KNOWLEDGE GRAPH

- ✓ The KB shows PostgresDatabase already instantiates an S3Bucket for backups.
- ✓ Dev does nothing - the backup already exists, correctly tagged and composed.
- ✓ No duplicate, no raw resource, no drift.

THE QUERY THAT ANSWERS IT

```
SELECT c2.name, e.relationship, e.note
FROM component_edges e
JOIN infra_components c1 ON e.from_id = c1.id
JOIN infra_components c2 ON e.to_id = c2.id
WHERE c1.name = 'PostgresDatabase';
```

→ PostgresDatabase --instantiates→ S3Bucket (backup bucket, automatic)

Why TiDB: The composition relationship lives in the graph, not buried in TypeScript. A flat repo search or a vector store of code snippets can't tell you 'this library already creates that resource for you.'

Blast radius before a refactor

claude-code

HEADLINE

Task: Change the naming scheme inside the S3Bucket library.

WITHOUT THE GRAPH

- ✗ Dev can't see everything that composes S3Bucket across environments.
- ✗ Ships the change, CI is green, prod static-assets + both analytics DBs break.
- ✗ The RDS breakage is the surprise - backups compose S3Bucket two hops away.

WITH TiDB KNOWLEDGE GRAPH

- ✓ Recursive CTE returns the full transitive closure: 7 dependents across prod + staging.
- ✓ Dev sees the RDS instances are in the blast radius before touching anything.
- ✓ Stages the change behind a flag, migrates per-environment, zero surprises.

THE QUERY THAT ANSWERS IT

```
-- What breaks if 'S3Bucket' changes? (upstream blast-radius traversal)
WITH RECURSIVE blast(from_id, to_id, relationship, depth) AS (
  SELECT e.from_id, e.to_id, e.relationship, 1
  FROM component_edges e
  JOIN infra_components c ON e.to_id = c.id
  WHERE c.name = 'S3Bucket'
  UNION ALL
  SELECT e.from_id, e.to_id, e.relationship, b.depth + 1
  FROM component_edges e
  JOIN blast b ON e.to_id = b.from_id
  WHERE b.depth < 10
)
SELECT DISTINCT b.depth, cf.name AS from_name,
  b.relationship, ct.name AS to_name,
  cf.component_type, cf.environment
FROM blast b
JOIN infra_components cf ON b.from_id = cf.id
JOIN infra_components ct ON b.to_id = ct.id
ORDER BY b.depth, from_name;

→ depth 1: PostgresDatabase, acme-prod-data-exports, acme-prod-static-assets, acme-staging-data-exports
depth 2: acme-prod-analytics-db, acme-staging-analytics-db (via PostgresDatabase), acme-prod-assets-dns (fronts static-assets)
```

Why TiDB: A recursive CTE traverses arbitrary-depth dependency chains in one query. Vector similarity finds 'related-looking' code; it can't compute a transitive closure. This is graph reachability - exactly what SQL recursion is for.

Staging parity, picked up warm across tools

claude-code

Task: Bring staging to parity with production (DNS + SSO).

WITHOUT THE GRAPH

- ✗ Each tool/dev re-discovers the prod pattern from scratch.
- ✗ Conventions drift: someone forgets proxied:true, someone hand-rolls the SSO redirect.

WITH TiDB KNOWLEDGE GRAPH

- ✓ Claude Code scaffolds static-assets + assets-dns, writes back to the KB.
- ✓ Codex opens cold, reads the session log, continues with admin-dns - no re-briefing.
- ✓ Cursor runs the dependency CTE, sees SSO must redirect to a DnsRecord, finishes it.
- ✓ Three tools, one shared memory, conventions preserved end to end.

THE QUERY THAT ANSWERS IT

```
SELECT developer, action, detail FROM session_log
ORDER BY created_at DESC LIMIT 5;
```

→ claude-code created acme-staging-static-assets, acme-staging-assets-dns ...

Why TiDB: The session log + component graph are shared state every tool reads and writes. The next session starts warm instead of re-deriving context from the filesystem.

In-product scenario cards. Each contrasts the failure without the graph against the corrected path with TiDB, and shows the exact query that answers it.

The duplicate backup bucket. Asked to "add backups" for a database, a tool greps, finds nothing, and creates a raw duplicate. The graph shows *PostgresDatabase* already *instantiates* a backup bucket - so the right action is to write nothing.

Blast radius before a refactor. Before changing the *S3Bucket* library, one recursive query returns the full set of dependents across every environment - including the RDS instances two hops away. The change is staged safely instead of breaking production.

How to run it

The repository ships everything: the TiDB-backed knowledge base, both MCP servers, per-tool configs, the dashboard, and a presenter's runbook.

- 1 **Configure TiDB.** `cp .env.example .env` and paste your TiDB Cloud Serverless credentials (Connect → General).
- 2 **Set up.** `./setup.sh` - installs dependencies, creates the schema and seeds the graph on TiDB, and generates MCP configs for Claude Code, Codex, and Cursor.
- 3 **Launch the dashboard.** `./demo.sh` → open `http://localhost:7001` (the screen you narrate).
- 4 **Open three terminals.** In each, run `claude`, `codex`, and `cursor-agent` from the repo. All three load both MCP servers automatically.
- 5 **Follow the prompts** in `DEMO.md` / `PRESENTER.md`. As the tools work, the dashboard updates live from TiDB.

The two MCP servers each tool connects to

SERVER	ORIGIN	WHAT IT PROVIDES
<code>tidb</code>	PingCAP official	<code>db_query</code> / <code>db_execute</code> - raw SQL, recursive CTEs, vector search
<code>tidb-infra-kb</code>	This repo	<code>query_knowledge_base</code> + <code>write_component</code> - convention-checked, atomic writes

Repository: github.com/stephenlthorn/mem9-ai-coding · Apache-2.0. The dashboard falls back to local SQLite if no TiDB credentials are present, so the demo runs even with no network.